

The Westgate Website

The Database structure and feature:

So the database chosen for this project is Supabase (PostgreSQL) for the following reasons:

- **PostgreSQL-based architecture**
 - Supabase is built directly on PostgreSQL, providing a relational database with strong guarantees around data integrity, constraints, indexing, and transactional consistency.
 - The schema, queries, and migrations are fully portable to any standard PostgreSQL environment, avoiding vendor lock-in.
- **Full control over data modelling**
 - All tables, relationships, constraints, and indexes are explicitly designed and managed rather than abstracted away by the platform.
 - This ensures predictable behaviour and makes the data layer transparent and debuggable.
- **Row Level Security (RLS) enforced at the database layer**
 - Access control is implemented directly in the database using RLS policies rather than relying solely on frontend or API-layer checks.
 - This provides defence-in-depth and prevents unauthorized data access even if an API route is misconfigured.
- **Separation of concerns across the stack**
 - Supabase is used strictly for data persistence and authentication, while business logic is handled in the Node/Express backend and UI logic in the Next.js frontend.
 - This keeps the system modular and allows individual components to be replaced or scaled independently.
- **Clear migration and scalability path**
 - Because the system is PostgreSQL-first, the backend can be migrated to a self-hosted or cloud-managed PostgreSQL instance if requirements change.
 - This allows the project to scale without requiring architectural rewrites.

The structure and tables:

For this project I have used these 11 tables in Supabase:

- admins
- audit-logs
- coaches
- games_cache
- highlights
- ladder_cache
- league_configs
- news
- profiles
- sponsors
- team_media

admins

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
email	No description	text	text	×	⋮
created_at	No description	timestamp with time zone	timestampz	✓	⋮

That is the admin structure in Supabase, this table is used for storing the admin users who first have to be Auth users in the Authentication tab in Supabase before being inserted manually into the admin table via the sql query within the sql editor section in supabase. I've thought about making it as an option in the admin app on the website but changed my mind since i want there only to be one admin account other than mine which is gonna be the clubs official email address and that is it no one else should be allowed therefore i didnt add that feature.

Policies:

I have only made one policy for the admin table:

Enable read access for all users	SELECT	public
----------------------------------	--------	--------

Ive done it this way because ive already got a security to who can actually access the admin part of the app that i didnt need to once again make this to only admins for i just kept it

simple and made it public, and since this table is only used in the ‘users page’ in the admin app which showcases the admin and user profiles.

audit-logs

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
created_at	No description	timestamp with time zone	timestamptz	✓	⋮
admin	No description	text	text	×	⋮
action	No description	text	text	×	⋮
target_id	No description	uuid	uuid	✓	⋮
details	No description	text	text	✓	⋮

That is the structure of the audit-logs table. The reasoning behind this table and its functionality to the website is to ensure accountability, every POST, PUT, and DELETE request from the Express backend is paired with an insert into the audit_logs table. This provides a permanent trail of editorial changes for the club's digital assets.

Policies:

The audit_logs table operates as a backend-only system table. Row Level Security is disabled because all access is mediated by secured Express API endpoints using the Supabase service_role key. No client-side access is permitted. This design ensures a tamper-resistant audit trail while avoiding redundant policy enforcement at the database layer.

coaches

Name	Description	Data Type	Format	Nullable
id	No description	uuid	uuid	×
created_at	No description	timestamp with time zone	timestampz	✓
name	No description	text	text	×
role	No description	USER-DEFINED	coach_role	×
team_slug	No description	USER-DEFINED	team_page	×
image_url	No description	text	text	✓
display_order	No description	integer	int4	✓

The above image showcases the coaches table structure which contains the usual stuff like id (uuid) and other stuff the role can only be either a head coach or an assistant role as pre defined in the USER_DEFINED to make it easier for the club, also thought about adding a physio or a team manager section but not for now maybe a later addition. Also the team_slug is also pre defined allowing 5 choices: senior men, reserve men, senior women, reserve women, junior. Which is basically all the teams that the Westgate club has these teams only so it only made sense

Policies:

coaches		
NAME	COMMAND	APPLIED TO
Allow admin all access	ALL	authenticated
Allow public read access	SELECT	public

For the coaches section ive made two policies:

- First admin accounts have access to all CRUD commands since i wanted to make the website perfect so that the club can manage the website themselves how they see fit and that i dont have to be involved other than solving some bugs if they ever come up

- Second is to allow public access for reading the coaches since i want to display the coaches of each team page i wanted to allow public access so that they can be displayed

games_cache

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
season_year	No description	integer	int4	×	⋮
team_name	No description	text	text	×	⋮
round	No description	text	text	✓	⋮
date_text	No description	text	text	✓	⋮
time_text	No description	text	text	✓	⋮
venue	No description	text	text	✓	⋮
w_score	No description	text	text	✓	⋮
opponent	No description	text	text	✓	⋮
a_score	No description	text	text	✓	⋮
last_updated	No description	timestamp with time zone	timestamptz	✓	⋮
config_id	No description	uuid	uuid	✓	⋮

The structure of the table which contains the games from different seasons is displayed above. The main function of this table is to store the games of the selected team's seasons, nothing more and nothing less. This table is populated by a scraping script explained in the backend part of this file.

Policies:

There is no policies for this table because just like the audit logs table it can only be changed by the admins in the admin account app but just like coaches it is displayed in the users page but that is it no other function is allowed in the user page therefore no policies were applied, (might be applied later on)

highlights

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
title	No description	text	text	×	⋮
youtube_id	No description	text	text	×	⋮
thumbnail_url	No description	text	text	✓	⋮
published	No description	boolean	bool	✓	⋮
created_at	No description	timestamp with time zone	timestampz	✓	⋮

The structure behind the highlights table is above, the idea behind the table is to have a youtube id or just the youtube link to the video that the club wants displayed and it will be displayed in the home page

Policies:

Same as the games reasoning behind the no policies rule, might be added later for extra security

ladder_cache

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
season_year	No description	integer	int4	×	⋮
team_name	No description	text	text	×	⋮
pos	No description	integer	int4	✓	⋮
team	No description	text	text	✓	⋮
p	No description	integer	int4	✓	⋮
w	No description	integer	int4	✓	⋮
d	No description	integer	int4	✓	⋮
l	No description	integer	int4	✓	⋮
f	No description	integer	int4	✓	⋮
a	No description	integer	int4	✓	⋮
gd	No description	integer	int4	✓	⋮
pts	No description	integer	int4	✓	⋮
last_updated	No description	timestamp with time zone	timestampz	✓	⋮
config_id	No description	uuid	uuid	✓	⋮

This table used for storing the ladder stats just like the games_cache table but for a league like wins losses goals for(f) goal difference (gd) ..., so inside the same scraping file is the scraping code for the ladder stats as well

Policies:

Same as the games_cache policies

league_configs

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
season_year	No description	integer	int4	×	⋮
team_name	No description	text	text	×	⋮
ladder_url	No description	text	text	×	⋮
fixtures_url	No description	text	text	×	⋮
is_active	No description	boolean	bool	✓	⋮
created_at	No description	timestamp with time zone	timestampz	✓	⋮

This is a table made to store all the links year of the season the team name and every specification used by the scraping file to know what and where from its scraping the data from and how to store the data in terms of the year and make sure its for the correct team

Policies:

There is no policies because this table is only used in the admin and in the backend files so i thought it was excessive to add unnecessary policies if its already protected by the security behind the admin app, so basically the admin has full access to the CRUD operations

news

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
title	No description	text	text	×	⋮
content	No description	text	text	×	⋮
published	No description	boolean	bool	✓	⋮
created_at	No description	timestamp with time zone	timestampz	✓	⋮
updated_at	No description	timestamp with time zone	timestampz	✓	⋮
image_url	No description	text	text	✓	⋮

This table is used to store all of the data for the news posts. It also includes boolean value named published this was added if a club wants to have a news post ready but don't want to make it public on the website.

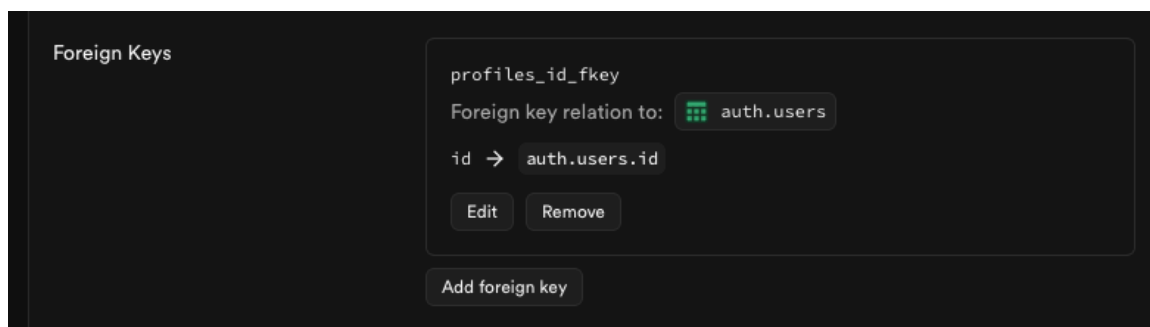
Policies:

Just like the games table it does not have any row level security because i thought it was just easier to make it public and only allow the admin accounts to have the tools for the CRUD operations and for the user UI i only use the news table for showcasing the data in it, meaning the SELECT operation but that is it

profiles:

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
created_at	No description	timestamp with time zone	timestampz	×	⋮
name	No description	text	text	✓	⋮
surname	No description	text	text	✓	⋮
logo	No description	text	text	✓	⋮
email	No description	text	text	✓	⋮

This table was made for the sole purpose of why not have it since i have the login page. Since I needed to have a log in page in order to login into the admin page ive made this table and the sign up page just to make it look like the login page is not looking odd, so I made the profiles table. Also:



(Id is a primary key from the Auth table)

Only people who are authenticated users by Supabase via the email confirmation, and after that the user can be inserted into this table. Meaning that not anyone can just signup with a made up email, and also this helps with the security and handling of passwords meaning that Supabase handles the security of passwords meaning even if the profiles table is

compromised their passwords will still be safe since the website doesn't hold or handle password its all done by Supabase, and the way a person is checked if its really them is by first seeing if the user is in the profiles table and then if it is it checkers their id which is fetched is the same as their Auth id which holds the db and then it checks the website to see if its the same one and the correct one and then the user is logged in into their profile.

Policies:

profiles			Disable RLS	Create policy	
NAME	COMMAND	APPLIED TO			
Admins can read all profiles	SELECT	authenticated			
Users can create their own profile	INSERT	public			
Users can read own profile	SELECT	public			
Users can update their own profile	UPDATE	public			

These are the policies for the profile table, the user has access to all CRUD operations for their own profile only, the DELETE will be added. The only edits the user can do is to change their profile image/logo, their name and surname and that is it.

The admin has access to all CRUD operations at all times because i wanted to make this website as automated as possible so that the club can manage it and not bug me for every little thing

sponsors

Name	Description	Data Type	Format	Nullable	
id	No description	integer	int4	×	
name	No description	character varying	varchar	×	
image_url	No description	text	text	×	
website_url	No description	text	text	×	
created_at	No description	timestamp without time zone	timestamp	✓	

Just like every football club in the world it has its sponsors, so I've made a table which holds the sponsor name, image_url and website_url if needed. Basically this table stored the data of the sponsors which are displayed in every page (other than admin) just above the footer as a part of the footer file.

Policies:

There is no policies for the sponsors table since the CRUD operations are handled in the admin part of the website and the only part that is done in the user UI is the SELECT which just showcases the sponsors and that is it so the extra RLS was not needed in my opinion, might be added later

team_media

Name	Description	Data Type	Format	Nullable	
id	No description	uuid	uuid	×	⋮
created_at	No description	timestamp with time zone	timestampz	×	⋮
url	No description	text	text	×	⋮
file_path	No description	text	text	×	⋮
team_slug	No description	text	text	×	⋮
uploaded_by	No description	uuid	uuid	✓	⋮

The team media table holds the images of each of the teams using the team_slug tag which has what team the images are intended for, this table stores all the needed image in the buckets which are going to be explained later therefore the file_path is there in the table but this table basically is used when displaying pictures for each of the teams

Policies:

Admins can manage media	ALL	authenticated
Public can view media	SELECT	public

The storage and buckets:

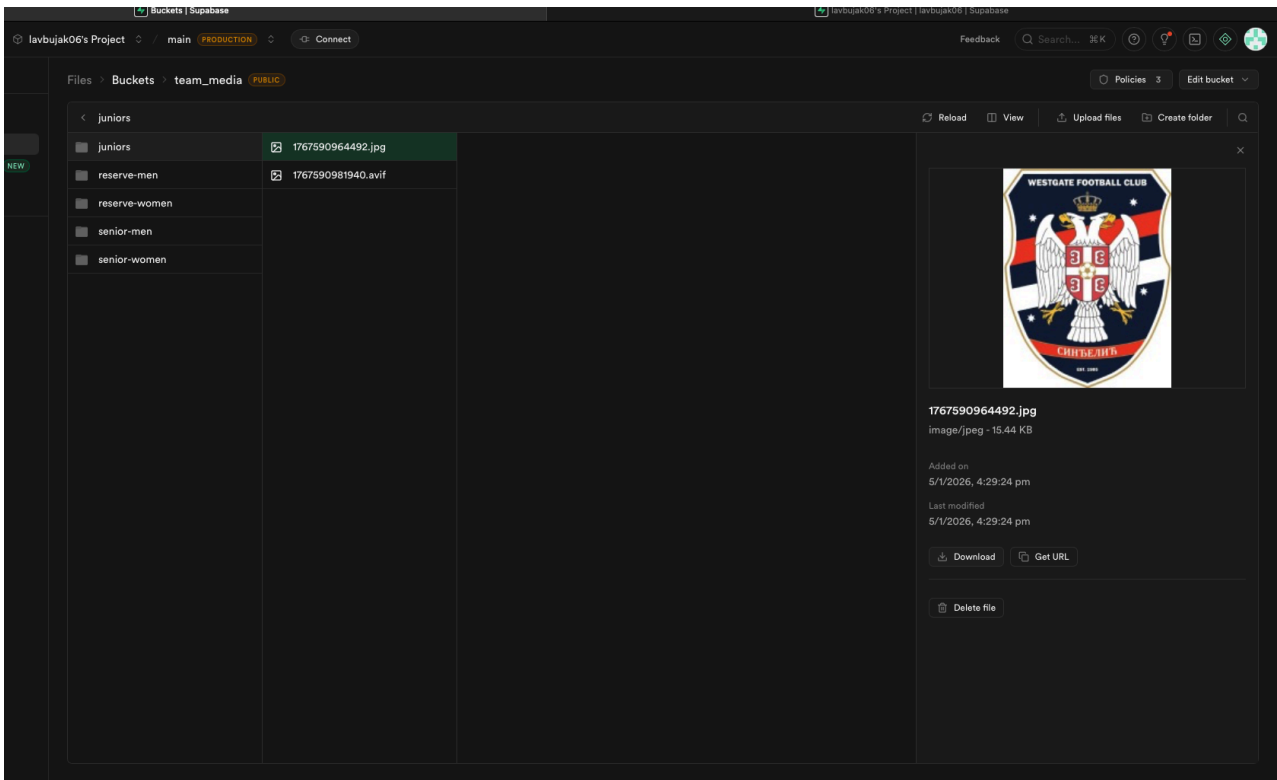
So because this website has images inside of it, in order to store the images in my DB Supabase has a thing called buckets which is dedicated for storing data like images.

For this project I've have so far made three different buckets with their own policies and functions:

- team_media
- coach_photos
- logos

team_media

This bucket holds all the photos for the teams (senior men, reserve men, senior women, reserve women, juniors) on this website. It is also structured like that as well because of the team_slug enum, therefore if an image is added to the juniors it is going to be added to that folder and be kept there



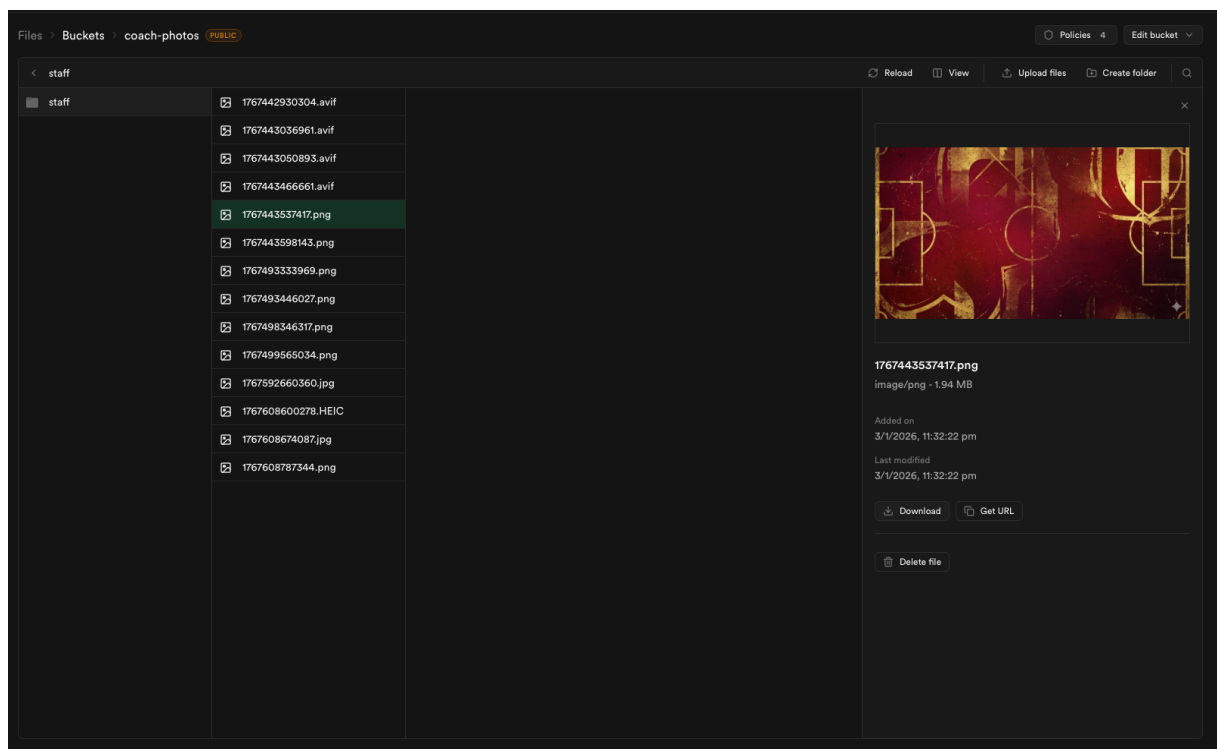
Policies:

TEAM_MEDIA (PUBLIC)			New policy
NAME	COMMAND	APPLIED TO	
Admin has access to everything 1fv44n6_0	SELECT	authenticated	
Admin has access to everything 1fv44n6_1	UPDATE	authenticated	
Admin has access to everything 1fv44n6_2	INSERT	authenticated	
Admin has access to everything 1fv44n6_3	DELETE	authenticated	

Admin has access to all of the CRUD operations as it is intended that way

coach_photos

The coach photos is designed exactly the same way as the team media but for coaches of course, but instead of having different folders for each team ive just grouped them into one folder called staff because there will be around maximum of 30 coaches and that is not that much therefore one folder is perfect for it



Policies:

The policies are the same like the team media, only the admin has the full access to all CRUD operations

COACH-PHOTOS PUBLIC New policy		
NAME	COMMAND	APPLIED TO
Allow admins to Upload and Update 1dzgw0k_0	INSERT	authenticated
Allow admins to Upload and Update 1dzgw0k_1	UPDATE	authenticated
Allow admins to Upload and Update 1dzgw0k_2	SELECT	authenticated
Allow admins to Upload and Update 1dzgw0k_3	DELETE	authenticated

logos

This bucket is intended to hold the images of the logos of the user accounts, for this the user has all control of their own user profile logo and access to all CRUD operations for them

LOGOS PUBLIC New policy		
NAME	COMMAND	APPLIED TO
Give users access to own folder 1peuqw_0	INSERT	public
Give users access to own folder 1peuqw_1	UPDATE	public
Give users access to own folder 1peuqw_2	SELECT	public
Give users access to own folder 1peuqw_3	DELETE	public

All three storage buckets in this project are configured as **public buckets**. This is intentional, as the files stored in these buckets (such as images and static assets) are required to be publicly accessible by the frontend for display on the website without requiring user authentication. Public access allows assets to be delivered efficiently and directly, improving performance and simplifying frontend integration. Despite being public, each bucket is protected by its own set of storage policies that strictly control **who can upload, update, or delete files**. These policies ensure that only authorised users can modify stored content, while public access is limited to read-only operations, maintaining both accessibility and security.

The "Backend Engine" (Node/Express Logic):

Folder structure:

backend:

- node_modules
- src:
 - middleware
 - routes
 - services
 - types
 - [server.ts](#)
 - [supabase.ts](#)
- .env
- Dockerfile

node_modules

The node_modules folder contains all third-party dependencies required at runtime and is generated from package.json. It's intentionally excluded from version control because it's environment-specific and reproducible.

The src folder:

This is the folder which holds everything for the backend.

middleware

The `middleware` folder contains request-level security logic that is executed **before protected backend routes are allowed to run**.

Its primary purpose is to **authenticate users, enforce authorization rules, and attach verified user context** to incoming requests.

Middleware functions act as a gatekeeper between the client and the application's core business logic.

Purpose of Middleware

- Enforces authentication and role-based access control
- Prevents unauthorised access to protected routes
- Centralises security logic to avoid duplication
- Provides verified user context to backend route handlers

Architectural Benefits

- Security-first design: Access control is enforced before business logic executes
- Separation of concerns: Routes handle functionality; middleware handles security
- Scalability: Additional roles can be introduced via new middleware
- Maintainability: Authentication logic is managed centrally

Middleware Deep Dive

Middleware functions in the backend act as **gatekeepers**: they process incoming requests before the route handlers execute. `requireAdmin` and `requireUser` are critical for **authentication and authorization**, ensuring only valid users can access protected resources.

requireAdmin.ts

Purpose

- Restricts access to routes that require **administrator privileges**.
- Ensures that only authenticated users who exist in the **admins** table can perform sensitive operations.

Step-by-Step Flow

Extract token from cookies

```
const token = req.cookies?.['sb-access-token'];
```

1.
 - The middleware expects the Supabase JWT (access token) to be stored in an HTTP-only cookie.
 - If no token is found → **401 Unauthorized** response (**Session missing**).

Verify JWT with Supabase

```
const { data: userData, error } = await  
supabase.auth.getUser(token);
```

2.
 - Sends the token to Supabase to validate it.
 - If invalid, expired, or missing → **401 Unauthorized** (**Invalid session**).

Check admin role in database

```
const { data: admin } = await  
supabase.from('admins').select('*').eq('id',  
userData.user.id).single();
```

3.
 - Queries the **admins** table to see if the authenticated user exists.
 - If not found → **403 Forbidden** (**Admins only**).

Attach user to request

```
req.user = userData.user;
```

4.
 - Saves the authenticated user object to **req.user** so downstream routes (like audit logs or route handlers) know **who is performing the action**.

Proceed to next middleware or route

```
next();
```

- 5.

- Only after passing all checks does the request continue to the route handler.

Why It's Important

- Prevents **unauthorized access** to admin routes.
 - Validates sessions using **JWTs from Supabase**, so no manual token verification is needed.
 - Centralizes **role-based access control**, keeping routes clean and secure.
 - Attaching `req.user` enables audit logging and better traceability of admin actions.
-

requireUser.ts

Purpose

- Ensures that the request comes from a **registered, authenticated user** (not necessarily an admin).
- Protects routes that require general user authentication, such as profile or account-related endpoints.

Step-by-Step Flow

Extract token from Authorization header

```
const token = req.headers.authorization?.replace('Bearer ', '');
```

1.
 - Expects a standard Bearer token in headers.
 - If missing → **401 Unauthorized** (No token provided).

Verify JWT with Supabase

```
const { data: userData } = await supabase.auth.getUser(token);
```

2.
 - Confirms the token is valid.
 - If invalid → **401 Unauthorized** (Invalid user).

Check user profile in database

```
const { data: profile } = await  
supabase.from('profiles').select('*').eq('id',  
userData.user.id).single();
```

3.
 - Confirms the user exists in the `profiles` table (i.e., they are registered in the system).
 - If not found → **403 Forbidden** (Not a registered user).

Attach user to request

```
req.user = userData.user;
```

4.
 - Makes the authenticated user available to the route handlers.

Proceed to next middleware or route

`next();`

- 5.

Why It's Important

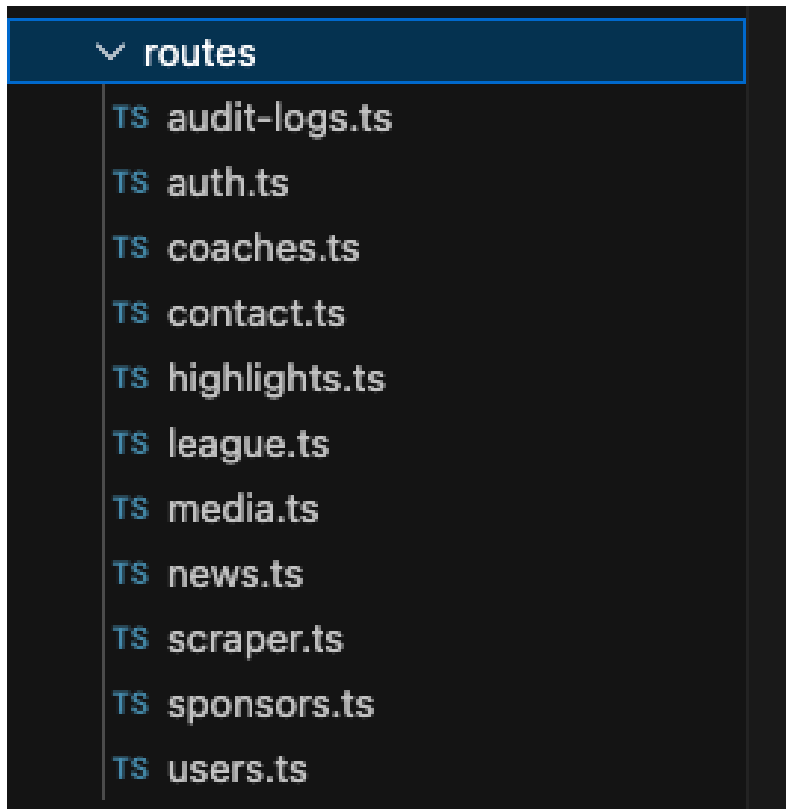
- Ensures **all requests are from valid, registered users**, reducing risk of unauthorized access.
 - Separates authentication logic from route handlers, keeping the backend modular.
 - Works seamlessly with `requireAdmin`, allowing a layered permission system (user vs admin).
 - Attaching `req.user` supports logging, auditing, and user-specific operations.
-

routes

The `routes` folder defines the **public API surface** of the backend. It contains all HTTP endpoints that the frontend and administrative tools interact with to read or modify application data.

Each route file groups related endpoints by feature or domain, creating a clear and structured interface between the client and the backend system.

routes folder structure:



Each of these files in the image above handles the API request for each dedicated feature/route

Purpose of the Routes Folder

- Defines how clients interact with the backend via HTTP requests
- Separates request handling from core application logic
- Provides a clear contract between frontend and backend
- Enables consistent application of middleware and access control

The routes layer is responsible for handling requests and responses, not enforcing security or managing database connections directly.

Role in the Request Lifecycle

A typical request flow follows this sequence:

1. Client sends an HTTP request to a defined route
2. Relevant middleware is executed (e.g. authentication or role checks)
3. Route handler processes the request and performs required actions
4. A structured response is returned to the client

This layered approach improves reliability, clarity, and security.

Organisational and Architectural Benefits

- Separation of concerns
 - Routes handle HTTP logic, middleware handles security, and services or database layers handle data operations.
- Scalability
 - New features can be added by introducing new route files without affecting existing endpoints.
- Maintainability
 - Grouping routes by domain makes the backend easier to understand, debug, and extend.
- Consistent access control
 - Middleware can be applied at the route level to enforce permissions consistently across endpoints.
- Improved testability
 - Individual routes can be tested independently with predictable inputs and outputs.

Most of the files are just basic routes folder dedicated to the CRUD operations, but three routes files are very different and very important:

auth.ts Routes – Deep Dive (the most important route file)

The `auth.ts` file handles **user authentication, session management, and admin verification** for the backend. It works alongside the `requireAdmin` middleware to enforce **role-based access control** and manage **secure sessions using HTTP-only cookies**.

Key Responsibilities

- Authenticate users using Supabase credentials (email & password).
 - Store session information securely in cookies (`sb-access-token` and `session_issued_at`).
 - Verify the current user and session status.
 - Logout users by clearing cookies and session info.
 - Provide admin-only endpoints protected by `requireAdmin` middleware.
-

Login Flow (POST /login)

1. Receive credentials

```
const { email, password } = req.body;
```

- Users submit their email and password in the request body.

2. Authenticate with Supabase

```
const { data, error } = await supabase.auth.signInWithPassword({
  email, password });
```

- Supabase validates the credentials and returns a **session** and **user object** if successful.
- On failure → **401 Unauthorized** (Invalid credentials).

3. Check admin status

```
const { data: admin } = await
supabase.from('admins').select('id').eq('id',
data.user.id).single();
```

- Confirms if the user exists in the **admins** table.
- If yes, logs an **audit entry** for admin login in **audit_logs**.

4. Set session cookies

```
res.cookie('sb-access-token', data.session.access_token, { httpOnly:
true, secure, sameSite: 'strict', maxAge: SESSION_DURATION });
res.cookie('session_issued_at', Date.now().toString(), { httpOnly:
true, secure, sameSite: 'lax', maxAge: SESSION_DURATION });
```

- **sb-access-token**: HTTP-only cookie storing the Supabase JWT, used by **requireAdmin** to validate sessions.
- **session_issued_at**: Timestamp for enforcing **session expiry**.
- Cookies are **HTTP-only** (not accessible via frontend JS) and expire in **1 hour**.

5. Return user info

```
res.json({ user: { id, email, is_admin }, is_admin });
```

- Sends user ID, email, and admin status to the frontend.

Get Current User (GET /me)

1. Read cookies

```
const token = req.cookies?.['sb-access-token'];
```

```
const issuedAt = req.cookies?.session_issued_at;
```

- Retrieves JWT and session timestamp from cookies.
- 2. **Handle missing cookies**
 - If either cookie is missing → returns `{ user: null, profile: null }`.
 - Prevents **infinite login loops** on page refresh.
- 3. **Check session expiry**

```
if (currentTime - sessionStart > SESSION_DURATION)
```

- Clears cookies and returns `401` if session expired.
- 4. **Fetch user & profile**
 - Validates the token with `Supabase.getUser()`
 - Retrieves profile from `profiles` table.
 - Checks admin status from `admins` table.
- 5. **Return structured response**

```
res.json({ user: {...}, profile: {...} });
```

- Provides the frontend with user info, profile, and admin flag.

Logout (POST /logout)

- Clears both cookies (`sb-access-token` and `session_issued_at`).
- Ensures the **user is fully logged out** on the client-side.
- Optionally, can also revoke the Supabase session client-side for UI consistency.

Admin Endpoint (GET /admins)

- Protected by `requireAdmin` middleware → **only accessible to admins**.
- Fetches and returns the list of admins from the database.
- Demonstrates **role-based access control** in action.

How Cookies and Middleware Work Together

- On login, the **Supabase JWT** is stored in `sb-access-token`.
- `requireAdmin` reads this cookie, verifies the token with Supabase, and confirms the user is an admin.

- Only after these checks does the request proceed to **protected admin routes**.
 - `session_issued_at` tracks session duration → allows the server to **invalidate old sessions automatically**.
-

Benefits of This Approach

- Keeps tokens **secure** and inaccessible to frontend JS.
 - Separates **authentication** (Supabase) from **authorization** (admin check in DB).
 - Centralizes **session logic** and access control → keeps route handlers clean and predictable.
 - Provides a **consistent API** for frontend apps to check login state and admin privileges.
-

scraper.ts

- **What it does:**
Provides admin-only endpoints to fetch, process, and cache external sports data (ladder and games) into the backend database.
- **How it does it:**
 - Protected with `requireAdmin` middleware to ensure only admins can trigger scrapes.
 - Retrieves league configuration from Supabase.
 - Calls `scraperService` functions to fetch ladder and game data from external sources.
 - Updates the local database by deleting old cache entries and inserting fresh results.
 - Returns a summary of the updated cache to the admin.
- **Why it's important:**
Automates the process of keeping the website's data up-to-date with external sources, ensures consistency and accuracy of displayed information, and maintains a controlled, secure mechanism for admins to update the cache.

contact.ts

- **What it does:**
Handles incoming contact form submissions, processes optional attachments, and sends structured emails to the technical department.
- **How it does it:**
 - Uses **Express** for routing and **Multer** to handle file uploads in memory.
 - Configures a **Nodemailer transporter** to send emails via Gmail or another SMTP service.

- Performs a **honeypot check** to block automated bot submissions.
- Formats the email content in HTML, including user details, message, and optional attachments.
- Sends the email to a fixed recipient (technical/admin email) and returns success or error responses.
- **Why it's important:**
Provides a secure, reliable way for users to contact administrators or report issues, prevents spam through bot detection, and ensures all inquiries are professionally formatted and logged via email.

services

The `services` folder contains reusable backend logic that supports the routes but is independent of request handling or middleware. It centralizes business logic and utility functions to keep the backend modular and maintainable.

Folder Functionality

- Provides functions that perform core operations, like fetching and processing external data.
- Keeps the route files focused on handling HTTP requests and responses.
- Can be reused across multiple routes without duplicating code.

Example: `scraperService.ts`

- **What it does:**
Extracts ladder and game data from external sources (MyGameDay) and formats it for storage in the backend.
- **How it does it:**
Fetches HTML from external URLs and parses tables using `cheerio` to return structured JSON data.
- **Why it's important:**
Keeps scraping logic separate from routes, making the backend more modular, testable, and maintainable while ensuring accurate, up-to-date data for the website.

types

The `types` folder contains TypeScript type definitions that extend or modify existing types to suit the backend's needs. It ensures type safety and prevents TypeScript errors when custom properties are added to standard objects.

Folder Functionality

- Provides global type extensions for the backend.
- Ensures TypeScript recognizes additional properties on standard objects, like `req.user` in Express.
- Centralizes type definitions so they can be used consistently across the backend.

Example File

- **What it does:**
Extends the Express `Request` interface to include a `user` property, which stores the authenticated user object.
- **How it does it:**
Declares a global namespace extension using TypeScript's `declare global` syntax.
- **Why it's important:**
Prevents TypeScript compilation errors when middleware attaches `user` to `req`. It ensures type safety and consistent usage of `req.user` across routes and middleware.

server.ts

The `server.ts` file is the **entry point of the backend**. It configures and starts the Express application, sets up middleware, and mounts all route handlers to define the backend API.

File Functionality

- Initializes the Express application.
- Configures **global middleware**:
 - `cors` for cross-origin requests and credentials handling.
 - `express.json()` to parse JSON request bodies.
 - `cookie-parser` to read cookies for session management.
- Mounts all route files under their respective API paths (`/api/auth`, `/api/news`, `/api/contact`, etc.).
- Starts the server on a specified port and logs the running status.

Why it's important

- Serves as the central configuration for the backend.
- Ensures consistent middleware application across all routes.
- Defines how the frontend communicates with the backend API.
- Provides a single, structured entry point for starting the backend service.

supabase.ts

The `supabase.ts` file is responsible for **creating and exporting a Supabase client** that can be used throughout the backend to interact with the database, authentication system, and storage.

File Functionality

- Uses `createClient` from the Supabase JavaScript library to connect to your Supabase project.
- Reads `SUPABASE_URL` and `SUPABASE_KEY` from environment variables to establish a secure connection.
- Exports a single `supabase` instance that can be imported and used in routes, middleware, and services.

Why it's important

- Provides a **centralized, reusable connection** to Supabase, avoiding repeated client creation.
- Ensures **secure access** to the database, authentication, and storage via environment-based configuration.
- Simplifies backend development by allowing all modules to interact with Supabase using a single, consistent client.

The .env file:

The `.env` file is used to **store environment-specific configuration variables** outside of your codebase. It allows sensitive information and configuration settings to be managed securely and flexibly without hardcoding them in your source code.

Purpose

- Stores **sensitive data** such as API keys, database URLs, authentication tokens, or email credentials.
- Holds **environment-specific settings**, like backend URLs, ports, or debug flags, that may differ between development, staging, and production.
- Makes the project **portable**: other developers or deployment environments can configure their own `.env` without changing the code.

Why it's important

- **Security**: Prevents sensitive information from being committed to version control.
- **Maintainability**: Centralizes configuration in one place, making it easy to update credentials or settings.
- **Flexibility**: Allows the same codebase to run in different environments (local, staging, production) with different settings.
- **Best Practice**: Many libraries, including Supabase, Node.js, and Express middleware, rely on environment variables for secure configuration.

The Dockerfile:

A **Dockerfile** is a blueprint that defines **how to package your backend application into a Docker container**. It specifies the environment, dependencies, and instructions needed to run your app consistently across different machines or deployment environments.

Purpose

- Defines a **consistent runtime environment** for your backend, including Node.js, system libraries, and dependencies.
- Automates the **build and deployment process** by creating a container image that can run anywhere Docker is supported.
- Isolates your backend from the host system, preventing version or dependency conflicts.

Why it's important

- **Portability:** The backend can run consistently on any machine or cloud provider without additional setup.
- **Scalability:** Makes it easy to deploy multiple instances of the backend in production (e.g., in Kubernetes or Docker Compose).
- **Reproducibility:** Ensures that all developers and environments use the same Node.js version, dependencies, and configuration.
- **Security:** The container isolates your app from the host system, reducing the risk of conflicts or accidental modifications.
- **Efficiency:** Enables automated builds and deployment pipelines, reducing manual setup and error

The "Frontend Anatomy" (Next.js & UI):

The folder structure for the frontend part of the website:

frontend:

- app:
 - admin folder containing all admin pages
 - account folder containing the account page for user profiles
 - login folder containing the login page file
 - sign up folder containing the sign up page file
 - pages folder containing all of the pages required for the user website UI
 - page.tsx is the home page of the website
- components:
 - holds all of the components used within the files in admin and pages folders
 - To make the page code neater and easier to maintain if the component is used between multiple pages
- context
- lib
- public

The app folder:

account/page.tsx

The Purpose

This is the **User Profile Management** page. It allows authenticated members to view and update their personal identity within the app. It acts as the bridge between the user's local session and your Supabase database.

Key Functionality

- **State Synchronization:** It uses `useEffect` to grab existing user data (Name, Surname, Logo) from your global `UserContext` and pre-fill the form fields.
 - **Image Handling (Supabase Storage):** When a user uploads a new logo, the file is sent to the `logos` storage bucket. It uses `upsert: true` to overwrite old files and then retrieves a `publicUrl` to save in the user's database row.
 - **Data Persistence:** It updates the `profiles` table in Supabase, linking the changes specifically to the user's unique ID (`user.id`).
 - **Safety Guards:** It includes a **Loading State** and a **Profile Guard**. This prevents the app from crashing or showing incorrect error messages while the system is still verifying who the user is.
-

Why Is It Here?

1. **Personalization:** It gives users control over their digital presence within the club's platform.
2. **Data Integrity:** It ensures that the "display names" and "avatars" used across the site (like in the navbar or admin lists) stay current and accurate.
3. **Security Flow:** By requiring a valid `profile` and `user` object before rendering the form, it ensures that only logged-in, legitimate users can touch the database.

login/page.tsx

The Purpose

This is the **Authentication Gateway**. Its primary job is to verify user credentials and intelligently route them to the correct part of the application based on their permissions. It acts as the primary "checkpoint" for the entire system.

Key Functionality

- **Multi-Step Verification:**
 - **Identity Check:** Authenticates the user via Supabase Auth (`signInWithPassword`).
 - **Role Verification:** Queries a specific `admins` table to see if the user has elevated privileges.
 - **Backend Sync:** Contacts a local API (`/api/auth/login`) with `credentials: 'include'` to establish a secure session and fetch the full user profile.
 - **Global State Hydration:** It uses `setUser` and `setProfile` from your `UserContext` so that the rest of the app (like the Navbar and Account page) immediately knows who is logged in without refreshing.
 - **Intelligent Redirection:** * **Admins:** Automatically pushed to `/admin/dashboard`.
 - **Standard Users:** Pushed to the home page (`/`).
 - **Visual Feedback:** Manages `loginLoading` states to show a spinner and handles error messages (like "Invalid password") using a styled error box.
-

Why Is It Here?

1. **Centralized Security:** It ensures that no user enters the "inner circle" of the app without a valid token and a verified profile.
2. **Role Separation:** By checking the `admins` table during the login process, it keeps the administrative interface separated from the general user interface from the very first click.
3. **UX Consistency:** It bridges the gap between the database (Supabase) and your React State (Context), ensuring the UI stays in sync with the user's login status.

signup/page.tsx

The Purpose

This is the **User Onboarding Portal**. It handles the initial registration of new members into the system. It is the starting point of the "SignUp -> Approval" lifecycle, ensuring that users are correctly added to the authentication database before they can request profile access.

Key Functionality

- **Identity Registration:** It uses `supabaseClient.auth.signUp` to create a new entry in the managed `auth.userstable`. This handles the heavy lifting of password hashing and security.
 - **Email Verification Trigger:** Upon a successful form submission, it triggers Supabase's built-in email confirmation flow. The user is notified to check their inbox to "prove" they own the email address.
 - **Flow Control:** After a successful signup, it automatically redirects users to the `/login` page. This forces the user to re-authenticate properly once their email is confirmed, ensuring a clean session start.
 - **Visual Consistency:** Like the login page, it uses a "Tactical UI" (dark overlays, backdrop blurs, and high-contrast red accents) to maintain the brand's high-energy, secure aesthetic.
-

Why Is It Here?

1. **Lead Generation:** It is the primary way new members enter the digital ecosystem of the club.
2. **Security Foundation:** By using Supabase's `signUp` method, you ensure that you aren't storing plain-text passwords on your own server, keeping the app compliant with modern security standard

ADMIN folder

The admin application is built as a **Secure Single-Page Application (SPA)** layer within Next.js. It operates on a "Source of Truth" model where the UI acts as a controller for three distinct backends:

1. **Supabase Auth/Storage:** For identity and high-volume media.
 2. **Express.js API (:5001):** A custom middleware for sensitive database operations.
 3. **Puppeteer Scraper:** A background service triggered by the League module to fetch real-time stats.
-

Detailed Module Specifications

Core Infrastructure & Security

- **layout.tsx (Tactical Framework)**
 - **How it works:** Implements a **Higher-Order Component** pattern using `<AdminGuard>`. Before any child page renders, the system verifies the user's JWT (JSON Web Token) and checks for a corresponding entry in the `admins` database table.
 - **Key Detail:** Uses the `usePathname` hook to provide real-time navigation feedback, ensuring the admin always knows their location within the system hierarchy.
- **accounts.tsx (Member Directory)**
 - **How it works:** Performs a `Promise.all` fetch to synchronize data from both the Auth system and the custom `profiles` table.
 - **Primary API:** `DELETE /api/users/:id`.
 - **Important Function:** `handleUserDelete` – This is a multi-stage destructive action that removes the user's database profile and revokes their Supabase authentication credentials simultaneously.
- **audit-logs.tsx (Forensic Tracking)**
 - **How it works:** Connects to a dedicated `audit_logs` table that records every "Write" operation performed in the HQ.
 - **Primary API:** `GET /api/audit-logs`.

- **Important Function:** `generatePDF` – Uses `jsPDF-AutoTable` to transform raw JSON log data into a formatted, professional security document for club records.

Personnel & Asset Management

- **`coaches/` (Staff Command)**
 - **How it works:** Features a **Two-Tier Asset Pipeline**. Photos are uploaded directly to the `coach-photos` Supabase bucket, while the metadata (role, team slug) is sent to the Express API.
 - **Primary API:** `POST/PUT /api/coaches`.
 - **Important Function:** `groupedCoaches` – A `useMemo` hook that performs client-side data reshaping, instantly organizing staff into their respective team categories (e.g., Senior Men, Juniors) without additional server requests.
- **`gallery/page.tsx` (Media Vault)**
 - **How it works:** A modular gallery system where each team section is its own component (`GallerySection`). This allows for **Atomic Loading**, where one team's photos can load or upload without affecting others.
 - **Primary API:** `POST /api/media/upload` (multipart/form-data).
 - **Important Function:** `handleUpload` – Orchestrates a `FormData` stream that pipes match-day images into team-specific storage folders.

Editorial & Content Control

- **`highlights/` (Video Producer)**
 - **How it works:** Built with a **Regex-based Parser**. Admins can paste any YouTube URL (shortened, mobile, or desktop), and the system strips the noise to store only the 11-character Video ID.
 - **Primary API:** `PUT /api/highlights/:id` (for toggle publish).
 - **Important Function:** `handleTogglePublish` – A "Hot-Swap" function that instantly changes the `published` boolean, triggering a re-validation on the public home page gallery.
- **`news/` (Editorial Suite)**
 - **How it works:** Features a **Hybrid Upload Engine**. Admins can choose between `uploadMode: 'url'` (for external assets) or `'file'` (for local club photography).
 - **Primary API:** `POST /api/news`.
 - **Important Function:** `handleFileChange` – An asynchronous listener that handles the Supabase storage handshake and returns a public URL before the article is even saved.
- **`sponsors/page.tsx` (Partner Intelligence)**
 - **How it works:** A specialized CRUD interface for commercial partners. It uses `FileReader` for **Instant Image Previews**, allowing admins to see the logo's transparency and fit before deploying the partner to the site.
 - **Primary API:** `DELETE /api/sponsors/:id`.

Data Automation

- **league/page.tsx (League Control)**
 - **How it works:** This is the most complex data module. It manages **Scraper Configurations**. It stores the URLs for external league tables and fixtures.
 - **Primary API:** `POST /api/scraper/sync/:id`.
 - **Important Function:** `triggerSync` – This function acts as a remote trigger for the backend **Puppeteer engine**. When clicked, the backend visits the GameDay website, parses the HTML, and updates the local SQL database with current ladder standings.

Pages folder

This section of the documentation focuses on the **Client-Facing Core**, detailing the public endpoints and high-traffic pages where fans, players, and sponsors interact with the club's digital identity. These pages are optimized for performance, data visualization, and accessibility.

Public Module Specifications

ContactPage (Contact HQ)

- **The "How":** This is a high-security communication portal. It uses a **FormData Multipart Stream** to handle text and binary data (files) simultaneously. It includes a "Honeypot" field—a hidden input designed to trap automated spam bots without affecting the user experience.
- **Primary API:** `POST http://localhost:5001/api/contact`
- **Important Function:** `handleSubmit` – This function dynamically appends user data and file attachments (like Birth Certificates or Recruitment CVs) into a single payload, dispatching it to the backend for email routing or database storage.

MensPage (Senior Men's Hub)

This page is used as a skeleton for the reserve men, senior women, reserve women pages are exactly the same.

- **The "How":** A data-intensive dashboard that serves as a **Live Stats Aggregator**. It orchestrates multiple asynchronous fetches to pull league standings, match results, and team photography in parallel.
- **Primary API:** `GET /api/league/ladder/:configId` & `GET /api/league/games/:configId`
- **Important Function:** `lastFiveForm` – A logic processor that analyzes raw match scores (`w_score` vs `a_score`). It calculates a "Win/Loss/Draw" status and returns a color-coded visual array (e.g., Green for W) to provide fans with an instant overview of the team's momentum.

NewsPage (The News Hub)

- **The "How":** This page implements a **Faceted Content Strategy**. It splits the news into two visual hierarchies: a "Flash Slider" for high-impact stories and a "Standard Grid" for historical archives. It uses Supabase's `.eq('published', true)` filter to ensure that only finalized "Broadcast" content is visible to the public.
- **Primary API:** Supabase Direct Client (`select('*') from 'news'`)

- **Important Function:** `heroNews.map` – Powering the "Accordion Slider," this allows the user to hover over an article, triggering a CSS `flex` transition that expands the item to show a content summary, while compressing others—maximizing screen real estate without losing context.

ArticlePage (Individual Report)

- **The "How":** This page is the **Deep-Dive Reader**. It uses dynamic routing (`/news/[id]`) to fetch a single, specific record based on the URL parameter. It features a "Reading Time" estimator and an "Asset Float" layout.
- **Primary API:** `supabase.from('news').select('*').eq('id', id).single()`
- **Important Function:** `Math.ceil(article.content.split(' ').length / 100)` – A real-time utility that calculates the "Min Read" time. By dividing the total word count by an average reading speed, it sets user expectations before they begin the report.

There is also an about us page but that page is just hard coded meaning it wasn't worth mentioning in this report.

The components folder:

There are a lot of components in this folder and each of them have their own functionality, from match slider used for images, to the nav bar and the footer files. All of these files are very important towards helping this website flourish and look amazing as it does now but there is one file here which is used in every admin page and is very important towards the security of the admin part of the website and that is the `AdminGuard.tsx` file.

AdminGuard.tsx:

This documentation covers the **AdminGuard**, the most critical security component in your application's architecture. It acts as the "Bouncer" for the Admin Command Center, ensuring that sensitive management tools are physically impossible to access by the general public or standard registered users.

Component: `AdminGuard.tsx`

The Virtual Perimeter Security

1. Why was this file made?

In a full-stack application, authentication (being logged in) is not the same as **authorization** (having permission).

- **The Problem:** A standard user could sign up for your site and, if they knew the URL (e.g., `/admin/league`), they could attempt to access your match-day scrapers or delete news articles.
 - **The Solution:** The `AdminGuard` creates a mandatory security checkpoint. It was built to wrap around the entire Admin Layout, creating a "Private Network" feel where the page content does not even exist in the browser's memory until the user's identity is verified against a specific **Whitelist** in your database.
-

2. How it Works: The 3-Step Verification Logic

The component operates as a **Higher-Order Component (HOC)** that intercepts the rendering process. It follows a strict "Guilty until proven innocent" workflow:

Step A: Session Discovery

Instead of just checking if a user exists, it calls `supabaseClient.auth.getSession()`.

- **Technical Detail:** Fetching the session is faster and more reliable for the frontend than fetching the full user object, as it checks the local JWT (JSON Web Token) stored in the browser. If no session exists, the guard immediately triggers `router.replace('/login')`.

Step B: The "Whitelist" Cross-Reference

Being "Logged In" is only half the battle. The Guard then performs a specific database query:

```
JavaScript
const { data, error } = await supabaseClient
  .from('admins') // Target the specific admin table
  .select('id')
  .eq('id', session.user.id)
  .maybeSingle();
```

- **Logic:** It looks for the logged-in User's ID inside a restricted `admins` table.
- **The Filter:** Using `.maybeSingle()` is a strategic choice. It prevents the code from crashing if the user isn't found; instead, it simply returns `null`, allowing the Guard to gracefully redirect the intruder away.

Step C: Conditional Rendering

The component uses a `loading` state to prevent "Flash of Unprotected Content" (FOUC).

- While verifying, it shows a "Verifying Admin Identity..." screen.
- If **Authorized**, it renders `{children}` (the actual admin pages).

- If **Unauthorized**, it renders `null` and boots the user to the login screen.

3. Key Technical Features

Feature	Technical Implementation	Purpose
Atomic Redirection	<code>router.replace</code>	Unlike <code>router.push</code> , <code>replace</code> removes the admin URL from the browser history, so the user can't click "Back" to try and see the admin panel again.
Database Shielding	<code>.from('admins')</code>	Separates "Admin" logic from "User" logic. Even if your <code>users</code> table is compromised, an attacker must also be present in the <code>admins</code> table to gain entry.
Crash Protection	<code>try...catch</code>	If the database is down or the internet cuts out during the check, the Guard defaults to a "Lockdown" state (redirecting to login) rather than accidentally letting the user through.

Export to Sheets

4. Summary for the Report

The `AdminGuard` is the primary gatekeeper of the **Sindjelic HQ**. By wrapping the `AdminLayout` in this guard, we ensure that the "Command Interface" is not just hidden, but **digitally locked**. It transforms the application from a public website into a secure enterprise tool, ensuring that the integrity of the club's data, finances (sponsors), and match records remains uncompromised.

Context/UserContext.tsx:

This documentation covers the **UserContext**, the "Central Nervous System" of the application's state. While the **AdminGuard** is the bouncer for the private areas, the **UserContext** is the **Global Identity Manager** that handles who the user is across every single page of the website.

Component: **UserContext.tsx**

The Global State Hub

1. Why was this file made?

In a modern web application, many different components need to know if a user is logged in at the same time (e.g., the Navbar needs to show "Logout," the Contact page needs to auto-fill your email, and the Admin panel needs to verify your identity).

- **The Problem:** Without this file, every single page would have to manually ask Supabase "Who is logged in?" every time the page changed. This would be slow, redundant, and would cause the UI to flicker.
 - **The Solution:** The **UserProvider** wraps the entire app. It fetches the user's data **once** the app starts and stores it in a "Global Store." Any component, no matter how deep, can then "tune in" to this store to get instant access to the user's data.
-

2. How it Works: The Dual-Sync Architecture

This file is unique because it synchronizes data between two different sources: **Supabase Auth** (for security) and your **Local Express Backend** (for custom profile data).

Step A: The Initial Handshake

When a user visits the site, the **fetchUserProfile** function runs immediately:

1. **Check Supabase:** It verifies if there is a valid session token in the browser.
2. **Verify with Backend:** It calls **http://localhost:5001/api/auth/me**. This is a critical security step—it ensures that even if the browser *thinks* it's logged in, your server confirms the session is still valid.

Step B: Real-Time Listening (**onAuthStateChange**)

The component sets up a "Listener" that stays active as long as the tab is open.

- If the user logs out in one tab, the listener detects the `SIGNED_OUT` event and instantly clears the user data across the whole app.
- If a token expires and refreshes, it silently updates the user's state without refreshing the page.

3. Key Technical Features

Feature	Technical Implementation	Purpose
Persistence Fix	<code>setUser(supabaseUser)</code> (Line 46)	Ensures the UI doesn't "jump" or show a blank screen while waiting for the backend API to respond. It uses the Supabase user as a temporary placeholder.
Automatic Cleanup	<code>signOut()</code> on 401/Reject	If the backend determines the session is faked or expired, it forces a full logout to protect the system's integrity.
Memoization	<code>useCallback</code>	Wraps the fetch function so it isn't "re-created" every time the app re-renders, preventing infinite loops and saving memory.
Loading State	<code>loading</code> boolean	Prevents "Race Conditions" where a page tries to load user data before the fetch has actually finished.

Export to Sheets

4. Summary for the Report

The `UserContext` is the engine behind the **Personalized Experience** of the Westgate Sindjelic website. By centralizing identity logic:

1. **Security is tighter:** We have a single point of failure and a single point of truth for authentication.
2. **Performance is higher:** Components don't waste time re-fetching user data.
3. **UX is smoother:** Logged-in users stay logged in across refreshes, and their profile information (like their name or role) is available instantly for a premium, customized feel.

Lib folder:

1. `lib/supabaseClient.ts` (The Public Gateway)

- **What it is:** This is the "Front Door" connection used by the parts of your site that run in the user's browser.
- **Functionality:** It uses the **Anonymous Key**, which is safe for the public to see. It is specifically configured to manage the user experience, such as keeping a user logged in after they close the tab (`persistSession`) and automatically refreshing security tokens so they don't have to log in every 20 minutes.
- **Use Case:** Fetching public news, checking if a user is logged in, or letting a fan submit a contact form.

2. `lib/supabaseServer.ts` (The Private Vault)

- **What it is:** This is the "Back Door" connection that *only* runs on your secure server (Node.js/Express).
- **Functionality:** It uses the **Service Role Key** (the "Master Key"). This key bypasses all security rules (RLS), meaning it can read or delete anything in your database regardless of who is logged in.
- **Critical Rule:** This file is never sent to the browser. If a regular user got hold of this key, they would have total control over your entire database.
- **Use Case:** Background tasks like the league scraper, deleting user accounts, or managing sensitive audit logs.

Summary for Report:

These files implement a **Dual-Tier Connection Strategy**. The `supabaseClient` handles standard user interactions with limited permissions, while `supabaseServer` provides high-privilege access for administrative tasks, ensuring the system remains secure by strictly isolating sensitive credentials from the end-user.